

Model Performance Test

Date	Team ID	Project Name
30 September 2026	SWTID-2026-9981	PocketSmart AI: Your Smart Budget & Recommendation Assistant

Model Overview

S.No	Parameter	Details
1	Model 1: Expense Categoriser	TF-IDF text vectoriser with Logistic Regression (scikit-learn) that predicts one of 8 categories: Food, Travel, Bills, Shopping, Health, Entertainment, Education, Other.
2	Model 2: Recommendation and Chat Model	Google Gemini large language model accessed through its API; prompts contain the user's spending summary, remaining budget and matching catalogue items.
3	Training Data for Model 1	Team-prepared CSV of labelled expense descriptions (for example 'swiggy dinner', 'bus pass', 'electricity bill') with category labels; split 80 percent for training and 20 percent for testing.
4	Prompt Design for Model 2	System prompt fixes the role (budget advisor), currency (Rs.), tone (short and practical) and rule (never suggest spending above the remaining budget); user data is passed as structured text.
5	Guardrails	Low-confidence categories fall back to 'Other' and can be corrected by the user; LLM output is limited in length; failures show a friendly message and a rule-based tip is used instead.

Model Performance Evaluation Plan and Acceptance Criteria

S.No	Parameter	How It Is Measured	Acceptance Criteria
1	Categorisation Accuracy	Correct predictions divided by total predictions on the 20 percent test split	At least 85 percent
2	Precision, Recall and F1-score	Per-category classification report from scikit-learn	F1-score of at least 0.80 for each main category
3	Confusion Matrix	Reviewed to find commonly confused categories such as Shopping and Entertainment	Top confusions reviewed and training data improved
4	Categorisation Response Time	Time taken to return a category for one expense	About 2 seconds or less
5	Recommendation Relevance	Team members rate 20 sample recommendations as relevant or not relevant	At least 80 percent rated relevant
6	Budget Compliance of Suggestions	Check that every suggested item price is within the remaining budget	100 percent of suggestions within budget
7	Recommendation Response Time	Time from request to displayed suggestions	About 8 seconds or less
8	Robustness	Inputs with empty description, unusual spellings and very large amounts	No crash; clear message shown

Source Code Structure

File / Folder	Purpose
app.py	Main Streamlit application and page navigation
auth.py	Registration, login and password hashing
expenses.py	Add, edit, delete and import expenses
categoriser.py	Loads the trained model and predicts categories
budget.py	Category limits, percentage used and alerts
recommend.py	Builds spending summary and calls the Gemini API
data/expense_training.csv, data/catalog.csv	Training data and product / alternatives catalogue
requirements.txt	Python dependencies